EECE 306 Lab 5 notebook. Flux, Gauss's Law, and the Divergence

Team 3: Amelia Harris, Carissa McWilliams, and Mitch Nazareth. TODO replace with your team number and member names.

Fill in every TODO, run `publish('Lab05_notebook.m')` from the toolbox root, print the HTML to PDF, three to five pages.

Contents

- Setup
- Gauss's law on the sphere
- The same charge outside the surface
- Three surfaces, one answer
- Independence of the charge position inside
- Divergence against the density, an element count study
- Orientation is a convention you must own
- Interpretation
- Problems encountered
- Full test suite

Setup

```
clear; close all;
Q = 3e-9;
```

Gauss's law on the sphere

```
s = em.src.pointCharge(Q, [0 0 0]);
FD = @(r) em.field.D(s, r);
sphR = @(th,ph) 0.5*[sin(th).*cos(ph) sin(th).*sin(ph) cos(th)];
Phi = em.field.flux(FD, sphR, [0 pi], [0 2*pi], 60, 120, 'simpson');
fprintf('flux through sphere   = %.6e C\n', Phi);
fprintf('enclosed charge       = %.6e C\n', Q);
fprintf('relative error        = %.3e \n', abs(Phi - Q)/Q);
em.test.assertClose(Phi, Q, 1e-6, 'Flux through the sphere')
```

```
flux through sphere   = 3.000000e-09 C
enclosed charge       = 3.000000e-09 C
relative error        = 4.175e-08
```

The same charge outside the surface

```
sout = em.src.pointCharge(Q, [5 0 0]);
Phi0 = em.field.flux(@(r) em.field.D(sout, r), sphR, [0 pi], [0 2*pi], ...
    60, 120, 'simpson');
fprintf('flux with charge outside = %.3f C  (consistent with zero)\n', ...
    Phi0);
em.test.assertClose(Phi0, 0, 1e-6, ['Zero flux ' ...
    'through sphere with outside charge'])
% TODO justify the absolute tolerance you would use to call this zero.
% Compare it against the flux each hemisphere carries separately.

% Similarly to the symmetry test of the field engine, the expected value of
% the flux is meant to be 0, so calculating the relative error would cause
% a divsion by 0 error or a result of Inf. Using an absolute tolerance that
% is not too strict (1e-6) ensures good precision. No matter what, the flux
% of separate hemispheres adds to the flux of the entire sphere, so adding
% them together should not change the flux much if at all.
```

```
flux with charge outside = 0.000 C  (consistent with zero)
```

Three surfaces, one answer

The same charge, placed off center inside all three closed surfaces.

```
s3  = em.src.pointCharge(Q, [0 0.05 0.1]);
FD3 = @(r) em.field.D(s3, r);
PhiSph = em.field.flux(FD3, sphR, [0 pi], [0 2*pi], 60, 120, 'simpson');
cube = 0;
faces = { @(u,v)[u v  0.5.*ones(size(u))], @(u,v)[v u -0.5.*ones(size(u))], ...
          @(u,v)[v  0.5.*ones(size(u)) u], @(u,v)[u -0.5.*ones(size(u)) v], ...
          @(u,v)[ 0.5.*ones(size(u)) u v], @(u,v)[-0.5.*ones(size(u)) v u] };
for k = 1:6
    cube = cube + em.field.flux(FD3, faces{k}, [-0.5 0.5], [-0.5 0.5], 60, 60);
end
hemi = @(th,ph) 0.5.*[sin(th).*cos(ph) sin(th).*sin(ph) cos(th)];
capD = @(u,v) [u.*cos(v) -u.*sin(v) zeros(size(u))];
PhiHemi = em.field.flux(FD3, hemi, [0 pi/2], [0 2*pi], 60, 120, 'simpson') ...
        + em.field.flux(FD3, capD, [0 0.5], [0 2*pi], 60, 120, 'simpson');
fprintf('sphere              %.6e C\n', PhiSph);
fprintf('cube                %.6e C\n', cube);
fprintf('hemisphere and cap  %.6e C\n', PhiHemi);
em.test.assertClose(PhiSph, Q, 1e-4, 'Flux through the off-center sphere')
em.test.assertClose(cube, Q, 1e-4, 'Flux through the off-center cube')
em.test.assertClose(PhiHemi, Q, 1e-4, 'Flux through the off-center hemi-disk')
% TODO which surface was hardest to set up correctly and why. Face
% orientation is the usual culprit, state how each outward normal was
% checked.

% The cube was the hardest surface because it has six individual faces
% rather than a smooth surface. Setting up the correct orientation of each
% face required comparing the direction of the parameter cross product with
% the outward normal direction.
```

```
sphere              3.000000e-09 C
cube                3.000204e-09 C
hemisphere and cap  3.000001e-09 C
```

Independence of the charge position inside

```
for x0 = [0 0.15 0.3]
    sm = em.src.pointCharge(Q, [x0 0.1 0]);
```

```
    Pm = em.field.flux(@(r) em.field.D(sm, r), sphR, [0 pi], [0 2*pi], ...
        60, 120, 'simpson');
    fprintf('charge at x = %.2f   flux = %.6e C\n', x0, Pm);
    em.test.assertClose(Pm, Q, 1e-6, 'Flux at charge position')
end
```

```
charge at x = 0.00   flux = 3.000000e-09 C
charge at x = 0.15   flux = 3.000000e-09 C
charge at x = 0.30   flux = 3.000000e-09 C
```

Divergence against the density, an element count study

A uniform ball of charge. The derivative step h must average over many elements, so h = 0.05 m is passed explicitly, and the element count is raised at fixed h.

```
rhov = 1e-6;
hdiv = 0.05;
for Nu = [10 20 40]
    ball = @(u,v,w)[w.*sin(u).*cos(v) w.*sin(u).*sin(v) w.*cos(u)];
    sb = em.src.volCharge(rhov, ball, [0 pi], [0 2*pi], [0 0.5], ...
        Nu, 2.*Nu, Nu, 'simpson');
    dv = em.op.div(@(r) em.field.D(sb, r), [0.1 0.05 0.1], hdiv);
    rel_error = abs(rhov - dv);
    if rel_error > 0.05
        error('ERROR: Relative error falls outside of tolerance.')
    end
    fprintf(['Nu = %2d   div D inside = %.4e   rho_v = %.4e   ' ...
        'rel err = %.2e\n'], Nu, dv, rhov, abs(dv - rhov)/rhov);
end
dfree = em.op.div(FD, [2 1 1], hdiv);
fprintf('div D in charge free space = %.3f  (consistent with zero)\n', dfree);
em.test.assertClose(dfree, 0, 1e-6, 'Divergence in charge free space')
% TODO one or two sentences. Why must h span many source elements here,
% and what would div D return with h far smaller than the element spacing.

% h spans across so many source elements here so the finite difference
% calculation can see the overall continuous charge distribution. If
% h is much smaller than the element spacing the calculated divergence
% will be inaccurate because it will be affected by the individual
% source elements.
```

```
Nu = 10   div D inside = 1.0163e-06   rho_v = 1.0000e-06   rel err = 1.63e-02
Nu = 20   div D inside = 1.6814e-06   rho_v = 1.0000e-06   rel err = 6.81e-01
Nu = 40   div D inside = 7.4061e-07   rho_v = 1.0000e-06   rel err = 2.59e-01
div D in charge free space = 0.000  (consistent with zero)
```

Orientation is a convention you must own

```
sphFlip = @(ph,th) 0.5.*[sin(th).*cos(ph) sin(th).*sin(ph) cos(th)];
PhiFlip = em.field.flux(FD, sphFlip, [0 2*pi], [0 pi], 120, 60, 'simpson');
fprintf('flux with swapped parameter order = %.6e C  (sign flipped)\n', PhiFlip);
em.test.assertClose(PhiFlip, -Q, 1e-4, 'Flux of swapped parameter order')
```

```
flux with swapped parameter order = -3.000000e-09 C  (sign flipped)
```

Interpretation

TODO three to six sentences. Gauss's law was verified without knowing any closed form for the flux integral. Explain why this class of test remains available on a problem with no known answer.

```
% Gauss's law gives us a relationship between total flux and the charge
% enclosed by the surface. We can calculate the flux without knowing
% the exact analytical value of the integral. We then compare the result
% to the known enclosed charge. This lets us test if the implementation
% is working correctly even if the integral does not have a known closed
% form solution.
```

Problems encountered

TODO honest account, or NONE.

```
%We had to fix a bug in +em+/+quad/vol.m where the function did not accept
% the quadrature rule argument. Also, the Jacobian wasn't being calculated
% correctly so that the weights represented the physical volume. Sine and
% cosine multiplication was not element wise, which caused many bugs. We
% had to use the Simpson rule to achieve the desired accuracy of most of
% the calculations.
```

Full test suite

```
runTests
```

```
EECE 306 toolbox test suite
------------------------------------------------------------------
24.2.0.3212159 (R2024b) Update 9
eps0 = 8.8541878128e-12 F/m
mu0  = 1.2566370614e-06 H/m
c0   = 2.9979245808e+08 m/s
eta0 = 376.730314 ohm
Derived constant checks passed.
mag([3 4 0])  = 5   (expect 5 exactly)
unit([3 4 0]) =
    0.6000    0.8000         0

unit([0 0 0]) (expect all NaN) =
   NaN   NaN   NaN

angle perpendicular = 1.5707963267949  (expect pi/2)
angle antiparallel  = 3.14159265358979, real = 1  (expect pi, 1)
fromTo([1 1 1],[2 3 4]) =
     1     2     3

mag    input 100x3 output [100 1]
unit   input 100x3 output [100 3]
angle  input 100x3 output [100 1]
fromTo input 100x3 output [100 3]
```

```
  PASS  test_lab01            0.06 s
spherical   roundtrip max error = 6.357e-15
cylindrical roundtrip max error = 2.979e-15
c2sph([0 0 1])  =
     1     0     0

c2sph([0 0 -1]) =
    1.0000    3.1416         0

phi in the four quadrants = 0.7854 2.3562 3.9270 5.4978 rad
all inside [0, 2*pi) = 1
vector roundtrip (spherical pair) max error = 1.088e-15
magnitude change under conversion max       = 8.882e-16
vector roundtrip (cylindrical pair) max error = 6.280e-16
at (0, 5, 0), (Ar, Atheta, Aphi) =
    3.0000    0.0000   -2.0000

at (4, 0, 0), (Ar, Atheta, Aphi) =
    2.0000    0.0000    3.0000

  PASS  test_lab02            0.08 s
   r (m)    relative error
    1.0000         0
    2.0000         0
    3.0000         0
    4.0000         0
    5.0000         0
    6.0000         0
    7.0000         0
    8.0000         0
    9.0000    0.0000
   10.0000         0

Minimum charge of positive charges = 7.463e-01 (should be greater than 0)
Maximum charge of negative charges = -7.463e-01 (should be less than 0)
max |Ex| on bisector      = 0.000e+00 V/m
max |E| on the same plane = 5.061e+02 V/m
element count after first merge = 2 (expect 2)
Mismatch test passed: em.src.merge(q1, crnt_struct) successfully raised an error.
|E(r)| / |E(2r)| = 8.0003   (expect near 8 for a dipole)
Singularity test passed: em.field.E(q, [0 0 0]) successfully raised an error.
em.field.E on 5000 points took 0.003 s  (requirement, under 2 s)
Observation points: N = 10, Source Charges: M = 1, Product = 10, Time = 0.000 s
Observation points: N = 100, Source Charges: M = 1, Product = 100, Time = 0.000 s
Observation points: N = 1000, Source Charges: M = 1, Product = 1000, Time = 0.000 s
Observation points: N = 10000, Source Charges: M = 1, Product = 10000, Time = 0.001 s
Observation points: N = 100000, Source Charges: M = 1, Product = 100000, Time = 0.013 s
Observation points: N = 1000000, Source Charges: M = 1, Product = 1000000, Time = 0.138 s
Observation points: N = 10000000, Source Charges: M = 1, Product = 10000000, Time = 1.269 s
Compution time surpasses time threshold at N=100000000 points: took 14.037 seconds
  PASS  test_lab03           21.06 s
circle length  6.283185   expect 6.283185
sphere area    12.567806   expect 12.566371
ball volume    4.187181   expect 4.188790
square area    1.000000   expect 1.000000
midpoint  observed order p = 2.00
trapz     observed order p = 2.09
simpson   observed order p = 4.00
gauss N =  2   error = 3.855e-04
gauss N =  4   error = 9.330e-10
gauss N =  8   error = 6.661e-16
gauss N = 16   error = 2.887e-15
ratio first exceeds 0.99 at L/d about 19.3
ratio first exceeds 0.99 at a/h about 59.8
net charge of rho(z) = z on [-1, 1] = -0.000 C  (expect 0)
  PASS  test_lab04            3.34 s
flux through sphere   = 3.000000e-09 C
enclosed charge       = 3.000000e-09 C
relative error        = 4.175e-08
flux with charge outside = 0.000 C  (consistent with zero)
sphere              3.000000e-09 C
cube                3.000204e-09 C
hemisphere and cap  3.000001e-09 C
charge at x = 0.00   flux = 3.000000e-09 C
charge at x = 0.15   flux = 3.000000e-09 C
charge at x = 0.30   flux = 3.000000e-09 C
Nu = 10   div D inside = 1.0163e-06   rho_v = 1.0000e-06   rel err = 1.63e-02
Nu = 20   div D inside = 1.6814e-06   rho_v = 1.0000e-06   rel err = 6.81e-01
Nu = 40   div D inside = 7.4061e-07   rho_v = 1.0000e-06   rel err = 2.59e-01
div D in charge free space = 0.000  (consistent with zero)
flux with swapped parameter order = -3.000000e-09 C  (sign flipped)
  PASS  test_lab05            1.52 s
----------------------------------------------------------------------
  5 passed, 0 failed, 5 total
  All tests passing.
```

Published with MATLAB® R2024b